

Lecture 20 - Graphs and Basic Graph Algorithms

Computer Systems, Data Structures, and Data Management
(4CM508)

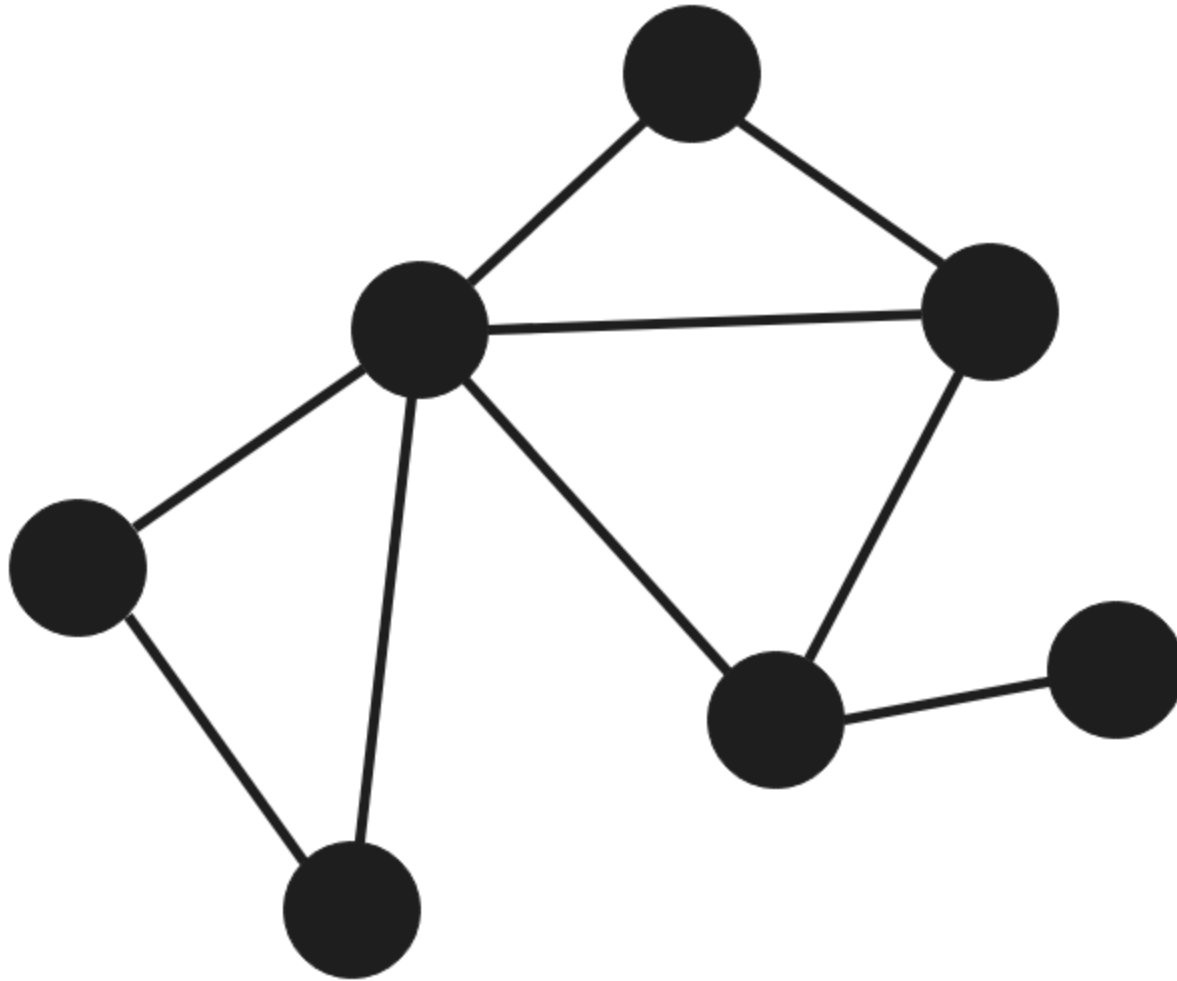
Dr Sam O'Neill

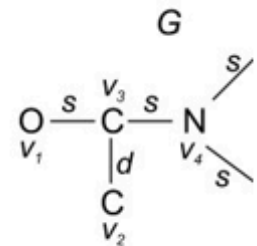
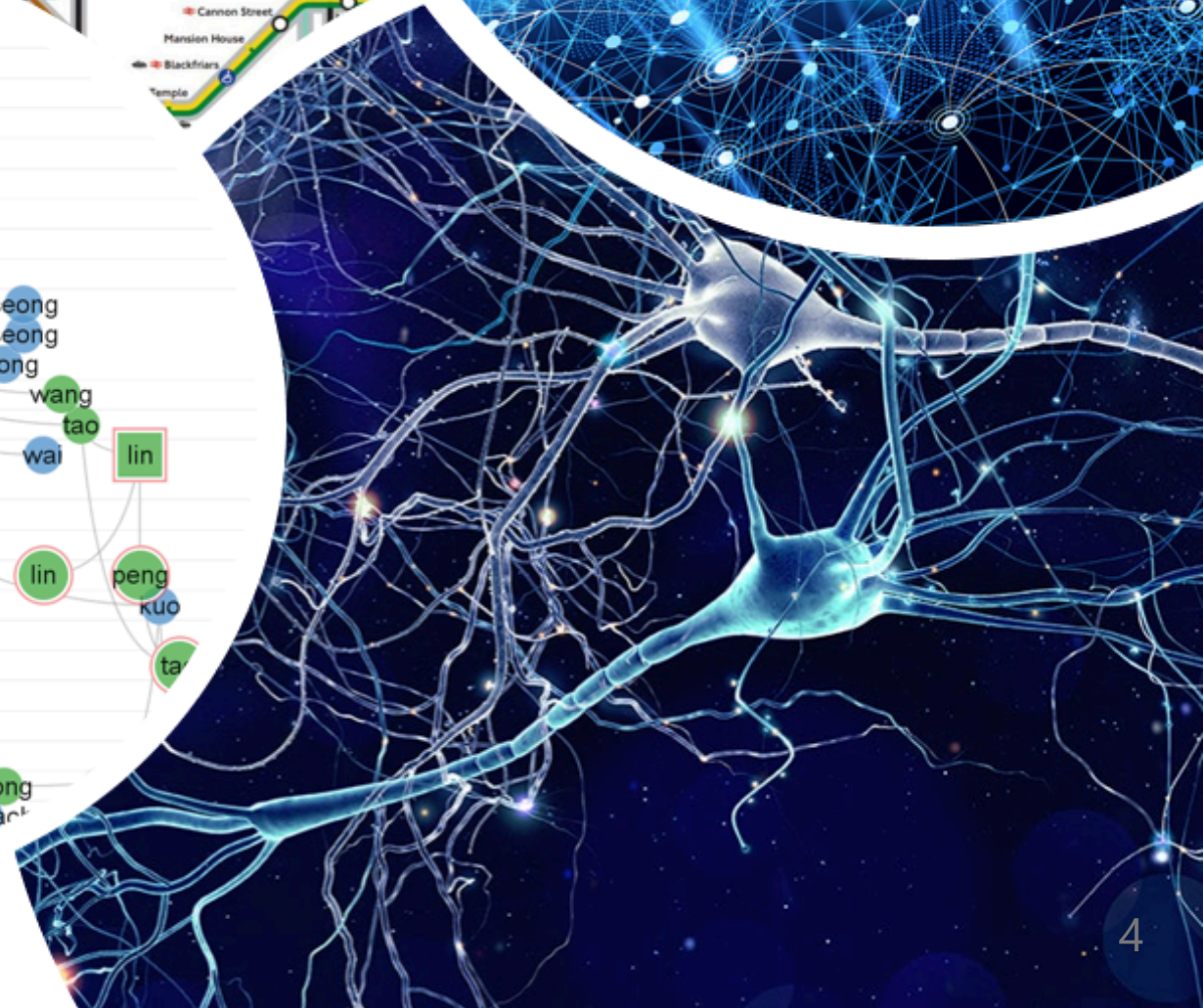
5 Minute Tutorials

I highly recommend you check out Michael Sambol's YouTube channel for quick concise videos on lots of things we have looked at on this course.

<https://www.youtube.com/@MichaelSambol>

What is a Graph?





Definition of a Graph

We will be dealing with **simple graphs** that contain **NO** loops (an edge joins a vertex to itself).

Undirected Graph

A undirected graph $G = (V, E)$ has a set of vertices V and a set of edges E .

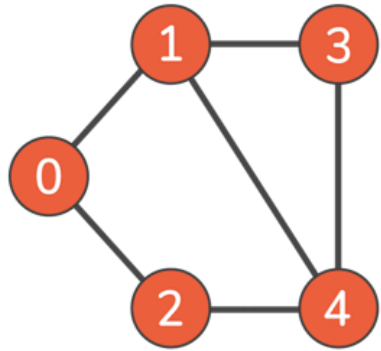
E is a subset of $V \times V$ where $\{u, v\} \in E$ such that $u \neq v$.

Directed Graph

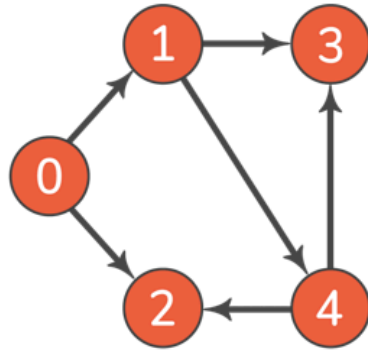
A directed graph $G = (V, E)$ has a set of vertices V and a set of edges E .

E is a subset of $V \times V$ where $(u, v) \in E$ such that $u \neq v$.

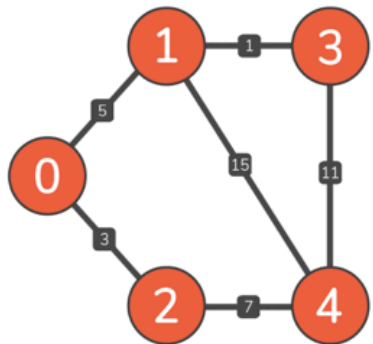
Examples



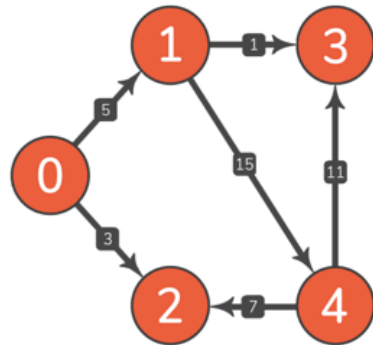
Undirected Graph



Directed Graph



Weighted Undirected Graph



Weighted Directed Graph

Why? What are they good for? (A Few Examples)

Network analysis: e.g. social, transportation, and communication networks.

Pathfinding and routing: Represent paths between nodes in a network, and algorithms can be used to find the shortest or most efficient paths between nodes.

Constraint satisfaction: Represent constraints and relationships between variables in a constraint satisfaction problem, and find solutions that satisfy all constraints.

Computer vision: Represent the structure of an image, with nodes representing pixels or regions and edges representing relationships between them.

Compiler optimization: Represent program control flow and data dependencies, and optimise the program's performance.

Classic Problems

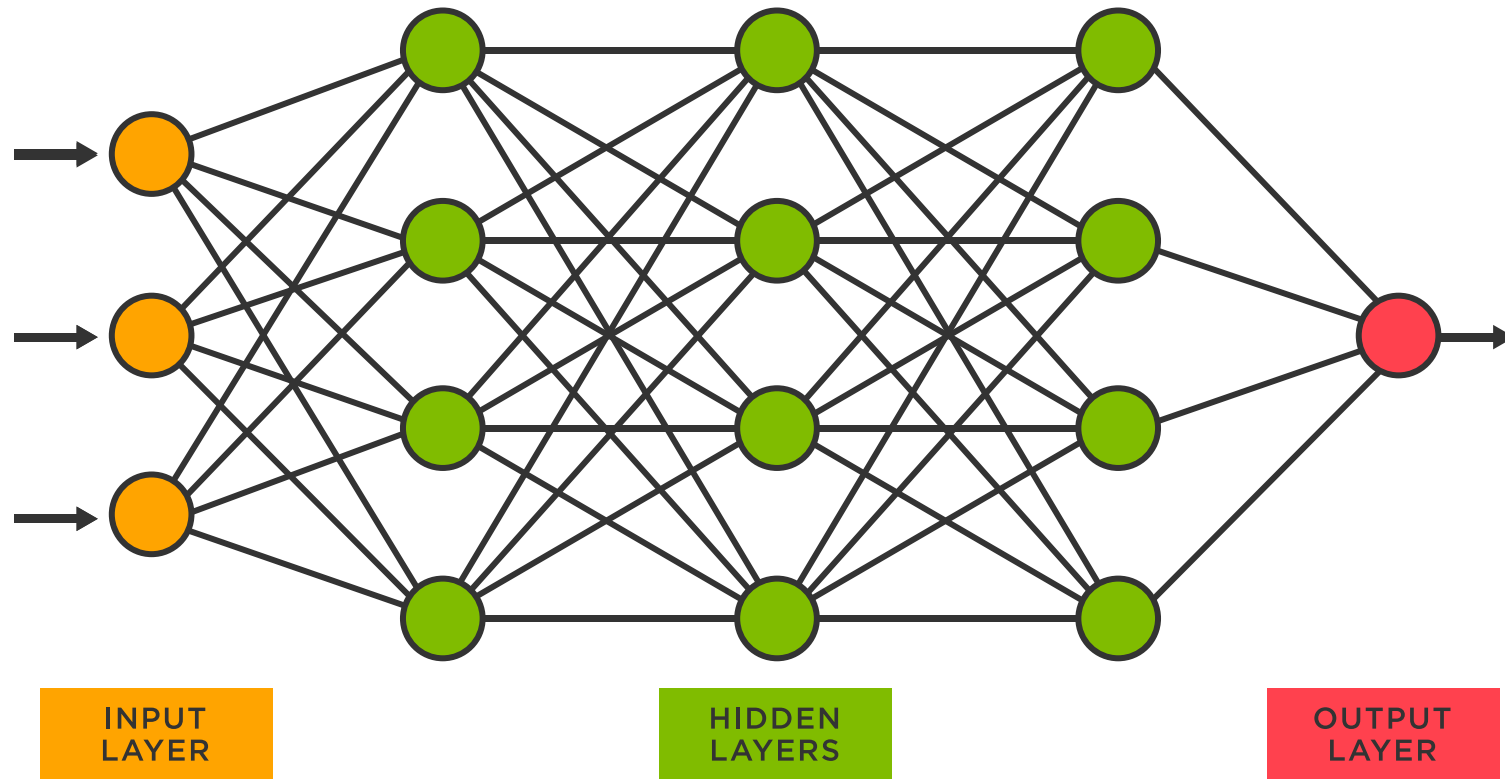
Here are some classic computer science problems involving graphs.

- Shortest Path Problem
- Minimum Spanning Tree Problem
- Travelling Salesman Problem
- Maximum Flow Problem
- Graph Colouring Problem

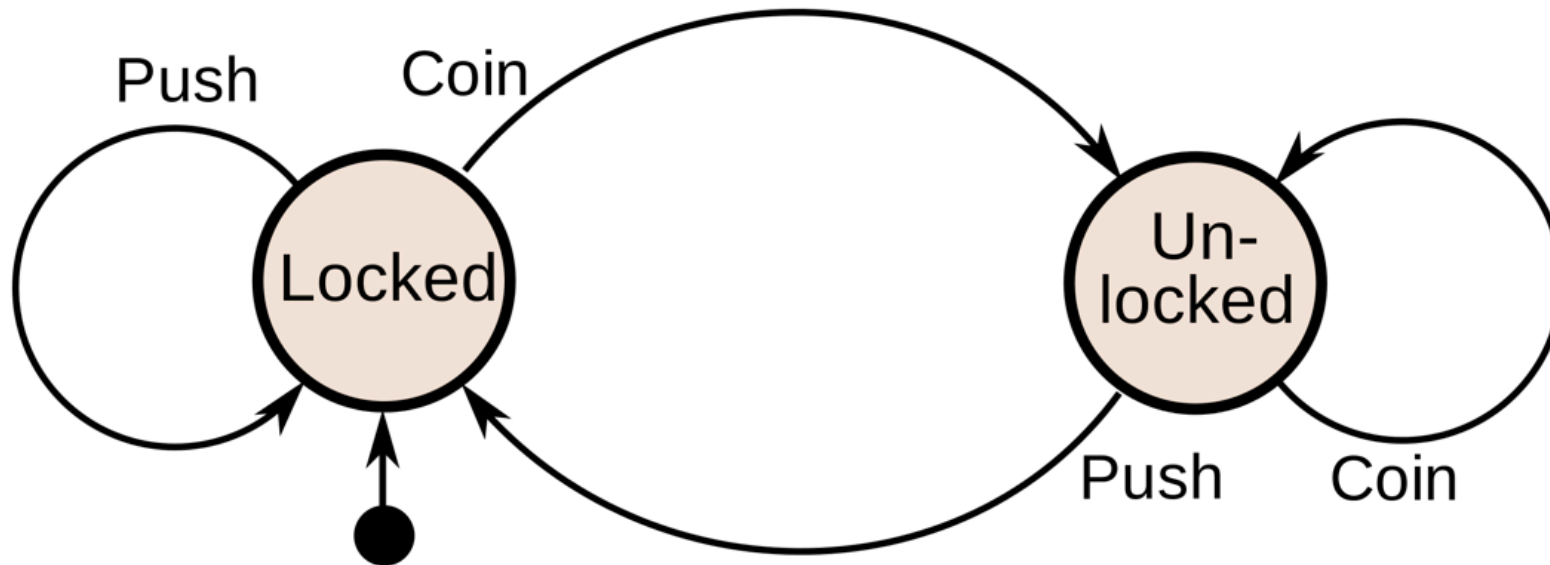
State Space of Chess and Go

State of the Art

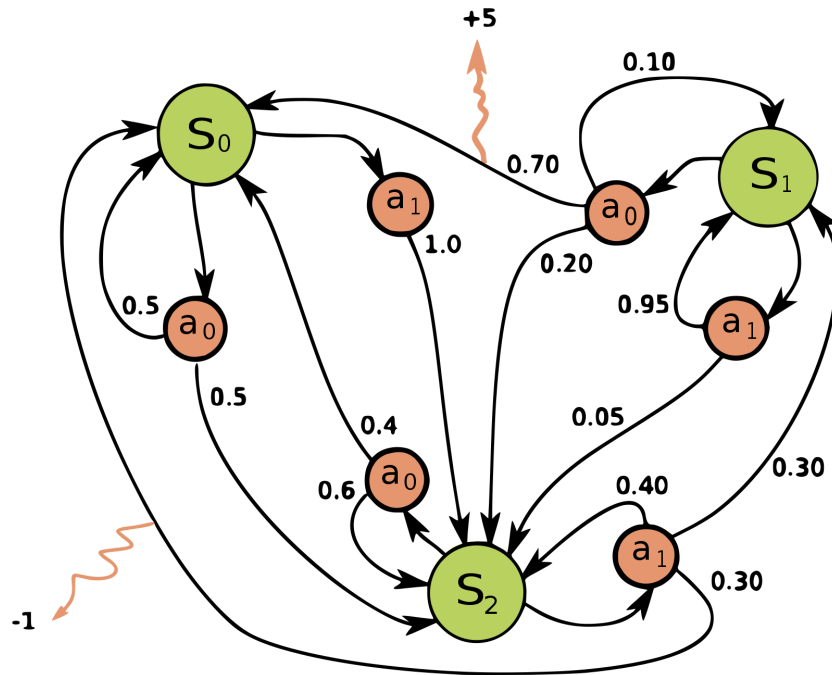
Neural Networks form the backbone of modern AI.



Finite State Machines



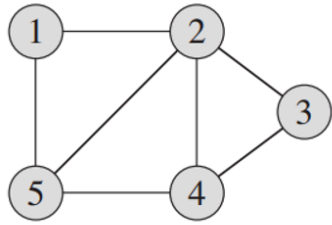
Markov Decision Processes



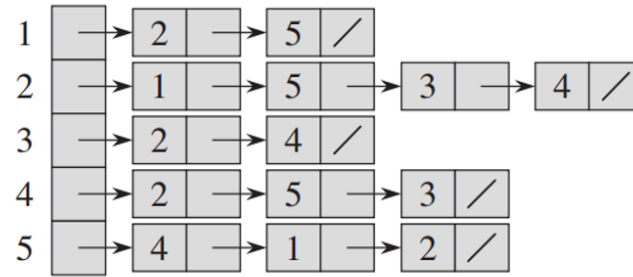
Forms the backbone of the theory underpinning Deep Reinforcement Learning -

<https://www.youtube.com/watch?v=Lu56xVIZ40M>

Representing a Graph in Memory



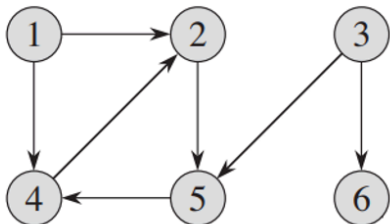
Undirected Graph



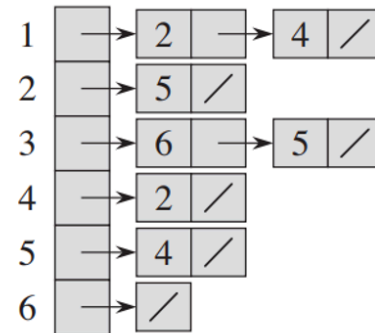
Adjacency List

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Adjacency Matrix



Directed Graph



Adjacency List

	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	0

Adjacency Matrix

Visualise Graphs

Please have a play with visualgo.net

Breadth-first Search

Search for a vertex in the graph - [Breadth-first Search in 4 Minutes - Youtube](#)

1. Create a set to keep track of visited vertices and a queue to keep track of vertices to be processed.
2. Add the starting vertex to the queue and mark it as visited.
3. While the queue is not empty, remove the first vertex from the queue and process it.
4. For each unvisited neighbor of the current vertex, add it to the queue and mark it as visited.
5. Repeat steps 3-4 until the queue is empty.

Visualise on a Larger Graph

Please have a play with visualgo.net

Breadth-first Search (Python Code)

```
from collections import deque

def bfs(graph, start):
    visited = set() # set to keep track of visited vertices
    queue = deque([start]) # create a deque and add the starting vertex to it
    visited.add(start) # mark the starting vertex as visited

    while queue: # loop until the queue is empty
        vertex = queue.popleft() # dequeue a vertex from the queue and print it
        print(vertex, end=" ")

        for neighbor in graph[vertex]: # get all adjacent vertices of the dequeued vertex vertex
            if neighbor not in visited: # if a adjacent has not been visited
                visited.add(neighbor) # mark it visited
                queue.append(neighbor) # enqueue it

# define the graph as a dictionary
graph = {
    0: [1, 2],
    1: [2],
    2: [0, 3],
    3: [2]
}

# start BFS traversal from vertex 2
print("BFS traversal starting from vertex 2:")
bfs(graph, 2)
```

How Many Edges in a Complete Graph?

- Please look at [here](#).
- What is the formula as a function of the vertices?
- Why?

Time and Space Complexity

Time Complexity

There are $|V|$ vertices and $|E|$ edges in the graph.

1. Each vertex is only visited once as it can only enter the queue once — $O(|V|)$
2. Every time a vertex is dequeued from the queue, all its k neighbors are explored and after all vertices are visited, we have examined all E edges — $O(|E|)$.

Thus time complexity is $O(|V| + |E|)$.

Space Complexity

We place all vertices in the visited set and queue once. Thus $O(|V|)$

Depth-first Search (Iterative)

Search for a vertex in the graph - [Depth-first Search in 4 Minutes - Youtube](#)

1. Create a set to keep track of visited vertices and a stack to keep track of vertices to be processed.
2. Push the starting vertex onto the stack and mark it as visited.
3. While the stack is not empty, pop the top vertex from the stack and process it.
4. For each unvisited neighbor of the current vertex, push it onto the stack and mark it as visited.
5. Repeat steps 3-4 until the stack is empty.

Depth-first Search - Iterative (Python Code)

```
# Define function for iterative DFS traversal using a stack
def dfs_iterative(graph, start):
    visited = [] # initialize an empty visited list
    stack = [start] # initialize a stack containing the start vertex
    while stack: # while there are vertices in the stack
        vertex = stack.pop() # pop the last vertex from the stack
        if vertex not in visited: # if the vertex is not visited yet
            visited.append(vertex) # add the vertex to visited list and print it
            print(vertex)
            for neighbour in reversed(graph[vertex]): # for all unvisited neighbours of the vertex, add them to the stack
                if neighbour not in visited:
                    stack.append(neighbour)

# Define the graph as a dictionary where keys are vertices and values are lists of neighbours
graph = {
    0: [1, 2],
    1: [2],
    2: [0, 3],
    3: [2]
}

# Print the DFS traversal starting from vertex 2 using all three functions
print("DFS traversal starting from vertex 2:")
dfs_iterative(graph, 2)
```

Visualise on a Larger Graph

Please have a play with visualgo.net

Depth-first Search (Recursive)

1. Create a set to keep track of visited vertices.

Starting from a given vertex, mark it as visited and process it.

2. For each unvisited neighbor of the current vertex, recursively call the DFS algorithm on that neighbor.
3. Repeat step 2 until all vertices have been visited.

Depth-first Search - Recursive (Python Code)

```
# Define function for recursive DFS traversal
def dfs(graph, start, visited=None):
    if visited is None: # if visited set is not provided, initialize it as an empty set
        visited = set()
    visited.add(start) # add the start vertex to visited set
    print(start) # print the start vertex
    for neighbour in graph[start]: # recursively call the function for all unvisited neighbours of start
        if neighbour not in visited:
            dfs(graph, neighbour, visited)

# Print the DFS traversal starting from vertex 2 using all three functions
print("DFS traversal starting from vertex 2:")
dfs(graph, 2)
```


Time and Space Complexity

Time Complexity

There are $|V|$ vertices and $|E|$ edges in the graph.

1. Each vertex is only visited once due to the fact that DFS tracks whether a vertex is visited — $O(|V|)$
2. Every time a vertex is visited, all its k neighbors are explored, after all vertices are visited, we have examined all E edges — $O(|E|)$

Thus time complexity is $O(|V| + |E|)$.

Space Complexity

We place all vertices in the visited set and stack once. Thus $O(|V|)$

Summary

- Graphs are everywhere, graphs are cool! Trust me!
- Represent with **adjacency list** or **adjacency matrix**
- **Breadth-first Search (BFS)**
 - Wide, fan out from current node to all other nodes
 - Time Complexity - $O(|V| + |E|)$
 - Space Complexity - $O(|V|)$
- **Depth-first Search (DFS)**
 - Follow a path as far as possible
 - Time Complexity - $O(|V| + |E|)$
 - Space Complexity - $O(|V|)$

References

Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. Introduction to algorithms. MIT press.

[Visualgo.net - Graphs](#)

[Michael Sambol - Youtube Channel](#)

[Two Minute Papers](#)